



Preface

How do we improve our knowledge of Python? Perhaps a more important question is “How do we show others how well we can write software in Python?”

Both of these questions have the same answer. We build our skills and demonstrate those skills by completing projects. More specifically, we need to complete projects that meet some widely-accepted standards for professional development. To be seen as professionals, we need to step beyond apprentice-level exercises, and demonstrate our ability to work without the hand-holding of a master crafter.

I think of it as sailing a boat alone for the first time, without a more experienced skipper or teacher on board. I think of it as completing a pair of hand-knitted socks that can be worn until the socks have worn out so completely, they can no longer be repaired.

Completing a project entails meeting a number of objectives. One of the most important is posting it to a public repository like SourceForge (<https://sourceforge.net>) or GitHub (<https://github.com>) so it can be seen by potential employers, funding sources, or business partners.

We'll distinguish between three audiences for a completed project:

- A personal project, possibly suitable for a work group or a few peers.
- A project suitable for use throughout an enterprise (e.g., a business, organization, or government agency)

- A project that can be published on the Python Package Index, PyPI (<https://pypi.org>).

We're drawing a fine line between creating a PyPI package and creating a package usable within an enterprise. For PyPI, the software package must be installable with the **PIP** tool; this often adds requirements for a great deal of testing to confirm the package will work in the widest variety of contexts. This can be an onerous burden.

For this book, we suggest following practices often used for “Enterprise” software. In an Enterprise context, it's often acceptable to create packages that are *not* installed by **PIP**. Instead, users can install the package by cloning the repository. When people work for a common enterprise, cloning packages permits users to make pull requests with suggested changes or bug fixes. The number of distinct environments in which the software is used may be very small. This reduces the burden of comprehensive testing; the community of potential users for enterprise software is smaller than a package offered to the world via PyPI.

Who this book is for

This book is for experienced programmers who want to improve their skills by completing professional-level Python projects. It's also for developers who need to display their skills by demonstrating a portfolio of work.

This is not intended as a tutorial on Python. This book assumes some familiarity with the language and the standard library. For a foundational introduction to Python, consider *Learn Python Programming, Third Edition*: <https://www.packtpub.com/product/learn-python-programming-third-edition/9781801815093>.

The projects in this book are described in broad strokes, requiring you to fill in the design details and complete the programming. Each chap-

ter focuses more time on the desired approach and deliverables than the code you'll need to write. The book will detail test cases and acceptance criteria, leaving you free to complete the working example that passes the suggested tests.

What this book covers

We can decompose this book into five general topics:

- We'll start with **Acquiring Data From Sources**. The first six projects will cover projects to acquire data for analytic processing from a variety of sources.
- Once we have data, we often need to **Inspect and Survey**. The next five projects look at some ways to inspect data to make sure it's usable, and diagnose odd problems, outliers, and exceptions.
- The general analytics pipeline moves on to **Cleaning, Converting, and Normalizing**. There are eight projects that tackle these closely-related problems.
- The useful results begin with **Presenting Summaries**. There's a lot of variability here, so we'll only present two project ideas. In many cases, you will want to provide their own, unique solutions to presenting the data they've gathered.
- This book winds up with two small projects covering some basics of **Statistical Modeling**. In some organizations, this may be the start of more sophisticated data science and machine learning applications. We encourage you to continue your study of Python applications in the data science realm.

The first part has two preliminary chapters to help define what the deliverables are and what the broad sweep of the projects will include.

Chapter 1, Project Zero: A Template for Other Projects is a baseline project. The functionality is a "Hello, World!" application. However, the additional infrastructure of unit tests, acceptance tests, and the use of a tool like **tox** or **nox** to execute the tests is the focus.

The next chapter, [***Chapter 2, Overview of the Projects***](#), shows the general approach this book will follow. This will present the flow of data from acquisition through cleaning to analysis and reporting. This chapter decomposes the large problem of “data analytics” into a number of smaller problems that can be solved in isolation.

The sequence of chapters starting with [***Chapter 3, Project 1.1: Data Acquisition Base Application***](#), builds a number of distinct data acquisition applications. This sequence starts with acquiring data from CSV files. The first variation, in [***Chapter 4, Data Acquisition Features: Web APIs and Scraping***](#), looks at ways to get data from web pages.

The next two projects are combined into [***Chapter 5, Data Acquisition Features: SQL Database***](#). This chapter builds an example SQL database, and then extracts data from it. The example database lets us explore enterprise database management concepts to more fully understand some of the complexities of working with relational data.

Once data has been acquired, the projects transition to data inspection. [***Chapter 6, Project 2.1: Data Inspection Notebook***](#) creates an initial inspection notebook. In [***Chapter 7, Data Inspection Features***](#), a series of projects add features to the basic inspection notebook for different categories of data.

This topic finishes with the [***Chapter 8, Project 2.5: Schema and Metadata***](#) project to create a formal schema for a data source and for the acquired data. The JSON Schema standard is used because it seems to be easily adapted to enterprise data processing. This schema formalization will become part of later projects.

The third topic — cleaning — starts with [***Chapter 9, Project 3.1: Data Cleaning Base Application***](#). This is the base application to clean the acquired data. This introduces the **Pydantic** package as a way to provide explicit data validation rules.

Chapter 10, Data Cleaning Features has a number of projects to add features to the core data cleaning application. Many of the example datasets in the previous chapters provide very clean data; this makes the chapter seem like needless over-engineering. It can help if you extract sample data and then manually corrupt it so that you have examples of invalid and valid data.

In **Chapter 11, Project 3.7: Interim Data Persistence**, we'll look at saving the cleaned data for further use.

The acquire-and-clean pipeline is often packaged as a web service. In **Chapter 12, Project 3.8: Integrated Data Acquisition Web Service**, we'll create a web server to offer the cleaned data for subsequent processing. This kind of web services wrapper around a long-running acquire-and-clean process presents a number of interesting design problems.

The next topic is the analysis of the data. In **Chapter 13, Project 4.1: Visual Analysis Techniques** we'll look at ways to produce reports, charts, and graphs using the power of **JupyterLab**.

In many organizations, data analysis may lead to a formal document, or report, showing the results. This may have a large audience of stakeholders and decision-makers. In **Chapter 14, Project 4.2: Creating Reports** we'll look at ways to produce elegant reports from the raw data using computations in a **JupyterLab** notebook.

The final topic is statistical modeling. This starts with **Chapter 15, Project 5.1: Modeling Base Application** to create an application that embodies lessons learned in the **Inspection Notebook** and **Analysis Notebook** projects. Sometimes we can share Python programming among these projects. In other cases, however, we can only share the lessons learned; as our understanding evolves, we often change data structures and apply other optimizations making it difficult to simply share a function or class definition.

In ***Chapter 16, Project 5.2: Simple Multivariate Statistics***, we expand on univariate modeling to add multivariate statistics. This modeling is kept simple to emphasize foundational design and architectural details. If you're interested in more advanced statistics, we suggest building the basic application project, getting it to work, and then adding more sophisticated modeling to an already-working baseline project.

The final chapter, ***Chapter 17, Next Steps***, provides some pointers for more sophisticated applications. In many cases, a project evolves from exploration to monitoring and maintenance. There will be a long tail where the model continues to be confirmed and refined. In some cases, the long tail ends when a model is replaced. Seeing this long tail can help an analyst understand the value of time invested in creating robust, reliable software at each stage of their journey.

A note on skills required

These projects demand a wide variety of skills, including software and data architecture, design, Python programming, test design, and even documentation writing. This breadth of skills reflects the author's experience in enterprise software development. Developers are expected to be generalists, able to follow technology changes and adapt to new technology.

In some of the earlier chapters, we'll offer some guidance on software design and construction. The guidance will assume a working knowledge of Python. It will point you toward the documentation for various Python packages for more information.

We'll also offer some details on how best to construct unit tests and acceptance tests. These topics can be challenging because testing is often under-emphasized. Developers fresh out of school often lament that modern computer science education doesn't seem to cover testing and test design very thoroughly.

This book will emphasize using **pytest** for unit tests and **behave** for acceptance tests. Using **behave** means writing test scenarios in the Gherkin language. This is the language used by the **cucumber** tool and sometimes the language is also called Cucumber. This may be new, and we'll emphasize this with more detailed examples, particularly in the first five chapters.

Some of the projects will implement statistical algorithms. We'll use notation like $\bar{x}$ to represent the mean of the variable x . For more information on basic statistics for data analytics, see *Statistics for Data Science*:

<https://www.packtpub.com/product/statistics-for-data-science/9781788290678>

To get the most out of this book

This book presumes some familiarity with Python 3 and the general concept of application development. Because a project is a complete unit of work, it will go beyond the Python programming language. This book will often challenge you to learn more about specific Python tools and packages, including **pytest**, **mypy**, **tox**, and many others.

Most of these projects use **exploratory data analysis (EDA)** as a problem domain to show the value of functional programming. Some familiarity with basic probability and statistics will help with this. There are only a few examples that move into more serious data science.

Python 3.11 is expected. For data science purposes, it's often helpful to start with the **conda** tool to create and manage virtual environments. It's not required, however, and you should be able to use any available Python.

Additional packages are generally installed with **pip**. The command looks like this:


```
% python -m pip install pytext mypy tox beautifulsoup4
```

Complete the extras

Each chapter includes a number of “extras” that help you to extend the concepts in the chapter. The extra projects often explore design alternatives and generally lead you to create additional, more complete solutions to the given problem.

In many cases, the extras section will need even more unit test cases to confirm they actually solve the problem. Expanding the core test cases of the chapter to include the extra features is an important software development skill.

Download the example code files

The code bundle for the book is hosted on GitHub at

<https://github.com/PacktPublishing/Python-Real-World-Projects>.

We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing/>. Check them out!

Conventions used

There are a number of text conventions used throughout this book.

CodeInText : Indicates code words in the text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. For example: “Python has other statements, such as `global` or `nonlocal`, which modify the rules for variables in a particular namespace.”

Bold: Indicates a new term, an important word, or words you see on the screen, such as in menus or dialog boxes. For example: “The **base**

case states that the sum of a zero-length sequence is 0. The **recursive case** states that the sum of a sequence is the first value plus the sum of the rest of the sequence.”

A block of code is set as follows:

```
print("Hello, World!")
```

Any command-line input or output is written as follows:

```
% conda create -n functional3 python=3.10
```

Warnings or important notes appear like this.

Tips and tricks appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: Email feedback@packtpub.com, and mention the book’s title in the subject of your message. If you have questions about any aspect of this book, please email us at questions@packtpub.com.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book we would be grateful if you would report this to us. Please visit <https://subscription.packtpub.com/help>, click on the **Submit**

Errata button, search for your book, and enter the details.

Piracy: If you come across any illegal copies of our works in any form on the Internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packtpub.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit <http://authors.packtpub.com>.

Share your thoughts

Once you've read *Python Real-World Projects*, we'd love to hear your thoughts! Please [click here to go straight to the Amazon review page](#) for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere? Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book, you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



<https://packt.link/free-ebook/9781803246765>

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly